

Mechanizm abstrakcji, klasy abstrakcyjne

Piotr Janowski

Abstrakcja

Pomyślcie o różnych pojazdach: samochód, rower, motocykl, hulajnoga elektryczna.

Co je łączy? A czym się różnią?

Wszystkie mają jakiś sposób poruszania się, ale każdy inny.

Abstrakcja to umiejętność patrzenia na obiekty przez pryzmat ich **najważniejszych cech**, pomijając szczegóły.

W programowaniu: opisujemy wspólne cechy grupy obiektów, nie wiedząc dokładnie, jak one działają.

Klasa abstrakcyjna

```
using System;
using System.Collections.Generic;

// Klasa abstrakcyjna - nie można tworzyć jej obiektów
abstract class Pojazd
{
    public string Nazwa { get; set; }

    public Pojazd(string nazwa)
    {
        Nazwa = nazwa;
    }

    // Metody abstrakcyjne - nie mają implementacji
    public abstract void Uruchom();
    public abstract void Zatrzymaj();

    // Metoda zwykła - może być wspólna dla wszystkich pojazdów
    public void WyszwietlNazwe()
    {
        Console.WriteLine($"Pojazd: {Nazwa}");
    }
}
```

abstract class Pojazd – nie można utworzyć obiektu typu Pojazd.

public abstract void Uruchom();
– metoda abstrakcyjna musi być nadpisana w klasach pochodnych.

Klasy pochodne – implementują metody abstrakcyjne

```
// Klasa pochodna - implementuje metody abstrakcyjne
class Samochod : Pojazd
{
    public Samochod(string nazwa) : base(nazwa) { }

    public override void Uruchom()
    {
        Console.WriteLine($"{Nazwa} odpala silnik benzynowy.");
    }

    public override void Zatrzymaj()
    {
        Console.WriteLine($"{Nazwa} hamuje i zatrzymuje się.");
    }
}

class Rower : Pojazd
{
    public Rower(string nazwa) : base(nazwa) { }

    public override void Uruchom()
    {
        Console.WriteLine($"{Nazwa} zaczyna jechać, gdy pedałujesz.");
    }

    public override void Zatrzymaj()
    {
        Console.WriteLine($"{Nazwa} zatrzymuje się po naciśnięciu hamulca.");
    }
}
```

Klasy Samochod i Rower dziedziczą po Pojazd i implementują metody abstrakcyjne.

Wywołanie

```
class Program
{
    static void Main()
    {
        List<Pojazd> pojazdy = new List<Pojazd>
        {
            new Samochod("Toyota Corolla"),
            new Rower("Merida")
        };

        foreach (var p in pojazdy)
        {
            p.WyswietlNazwe();
            p.Uruchom();
            p.Zatrzymaj();
            Console.WriteLine();
        }
    }
}
```

W **Main()** pokazujemy polimorfizm – traktujemy różne obiekty (Samochod, Rower) jako Pojazd.

Podsumowanie

Abstrakcja – pozwala skupić się na tym, co istotne, pomijając szczegóły.

Klasa abstrakcyjna – nie tworzymy jej obiektów; jest wzorcem dla klas pochodnych.

Metody abstrakcyjne – wymuszają implementację w klasach dziedziczących.

Polimorfizm – pozwala traktować różne obiekty w jednolity sposób.

Zalety i wady abstrakcji

Aspekt	Zalety	Wady
Projektowanie	Porządkuje system, wymusza logiczną strukturę	Może komplikować proste programy
Czytelność	Łatwiej zrozumieć ogólną ideę	Trudniej prześledzić szczegóły działania
Rozszerzalność	Łatwo dodać nowe klasy pochodne	Czasem trudniej zrozumieć zależności
Testowanie	Wymusza spójny interfejs	Klasy abstrakcyjne nie są testowalne bez pochodnych